Contents

Documentación de Diagramas de Estado — Aliflow 1
1. Orden 1
2. CodigoRetiro 1
3. Pago 4
4. EventoSincronizacion 4
5. Cartilla de fidelidad 5
Por qué estos 5 y no otros 6

Documentación de Diagramas de Estado — Aliflow

Cinco diagramas de estado, para los objetos del sistema con un ciclo de vida real (más de un estado posible y transiciones con condiciones).

---

1. Orden

Fuente: `../../uml/estado-orden.puml` · **Referencia:** UC4/UC5, `../../uml/diagrama-clases.puml`.

`COMPRADO → ENTREGADO` es el camino confirmado del flujo original. `COMPRADO → EXPIRADO` ocurre cuando vence el `CodigoRetiro` asociado. **Lo único que sigue abierto** es qué pasa con el **dinero** de una orden vencida: si se reembolsa, se pierde, o se le liquida al proveedor porque el almuerzo se preparó. El acta define la expiración, no la política de reembolso.

2. CodigoRetiro

Fuente: `../../uml/estado-codigo-retiro.puml` · **Referencia:** UC5, `../../uml/secuencia-retiro-entrega.puml`. | Estado | Significado | |—|—| | **VÁLIDO** | El pedido está disponible para ser retirado | | **UTILIZADO** | El Operador confirmó la entrega | | **VENCIDO** | Terminó el horario de retiro o terminó el día de la compra |

`VÁLIDO → UTILIZADO` es la transición atómica y condicional (`UPDATE... WHERE estado='VALIDO'`) que resuelve el riesgo R-14 (doble redención). `VÁLIDO → VENCIDO` dispara además la expiración de la `Orden` asociada — son dos objetos, pero una sola regla de negocio.

Regla de vigencia: el código vale **únicamente durante el día en que se hizo la compra**. `fechaExpiracion` se calcula como el mínimo entre la `horaMaximaRetiro` que configuró el local y el fin de ese día. Si se presenta otro día, debe mostrarse como vencido. Esto **cierra la decisión #5**, abierta desde el 27-jul.

Un efecto colateral que conviene notar: acotar la vigencia a un solo día reduce mucho el universo de códigos vigentes simultáneamente. Eso es justamente lo que hace viable exigir unicidad con solo 6 dígitos, y **refuerza la mitigación del riesgo R-15** sin que hubiera que hacer nada más.

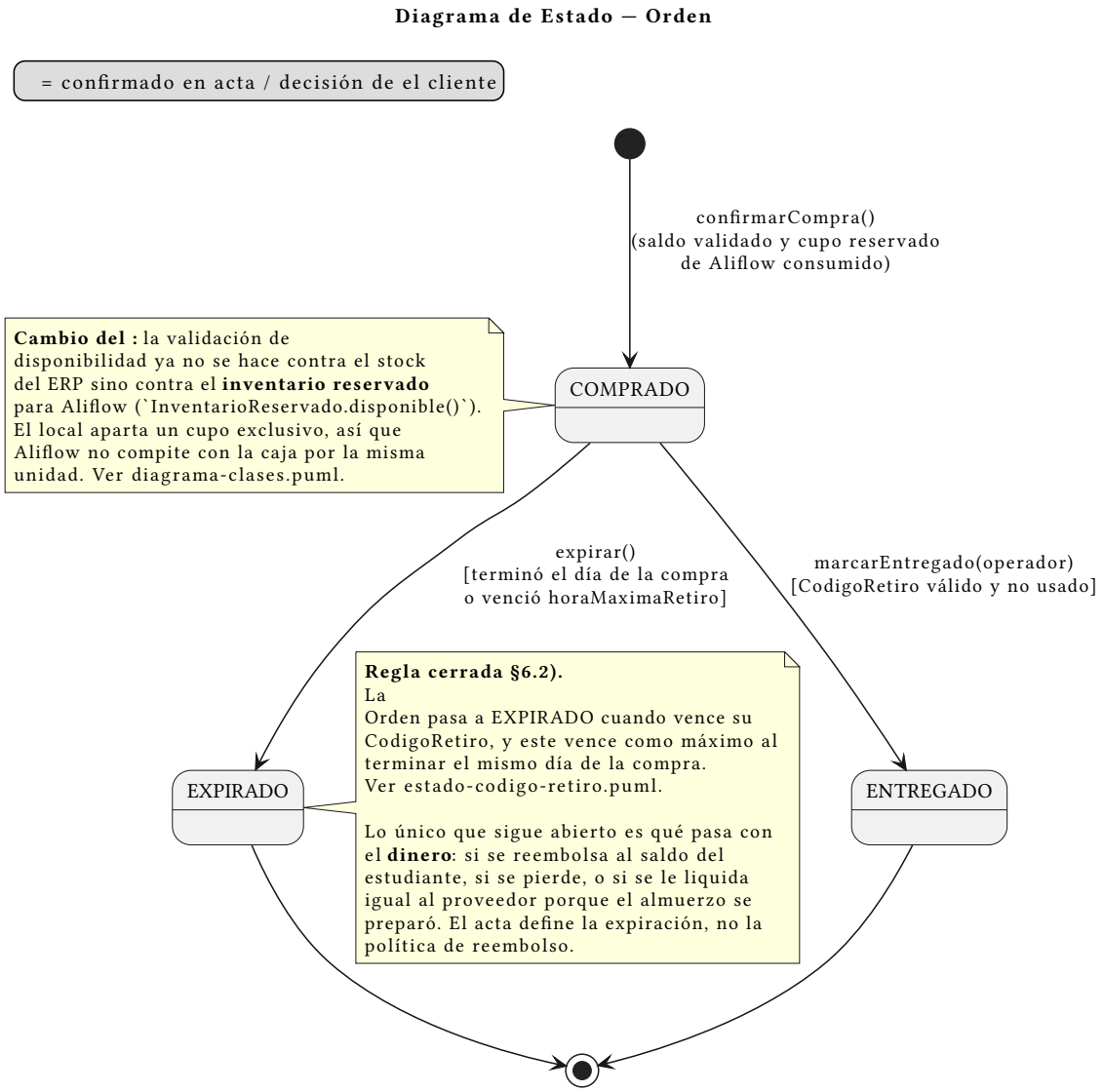


Figura 1: Estado: Orden

Formato del código: es un **código numérico corto de 6 dígitos**, porque el estudiante se lo dicta de viva voz al Operador y este lo digita. El formato no altera la máquina de estados, pero trae dos consecuencias de diseño que quedaron anotadas en el diagrama:

1. **Unicidad acotada, no global.** Seis dígitos son ~1 millón de combinaciones: alcanzan de sobra si la unicidad se exige solo entre los códigos **Vigentes de un mismo local**, pero no si se pretende que sean únicos históricamente. La generación reintenta ante colisión.
2. **Espacio de búsqueda pequeño.** Un código de 6 dígitos es adivinable por fuerza bruta de una forma que un UUID no lo era. Se registró como riesgo **R-15** (`../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`), con la mitigación correspondiente.

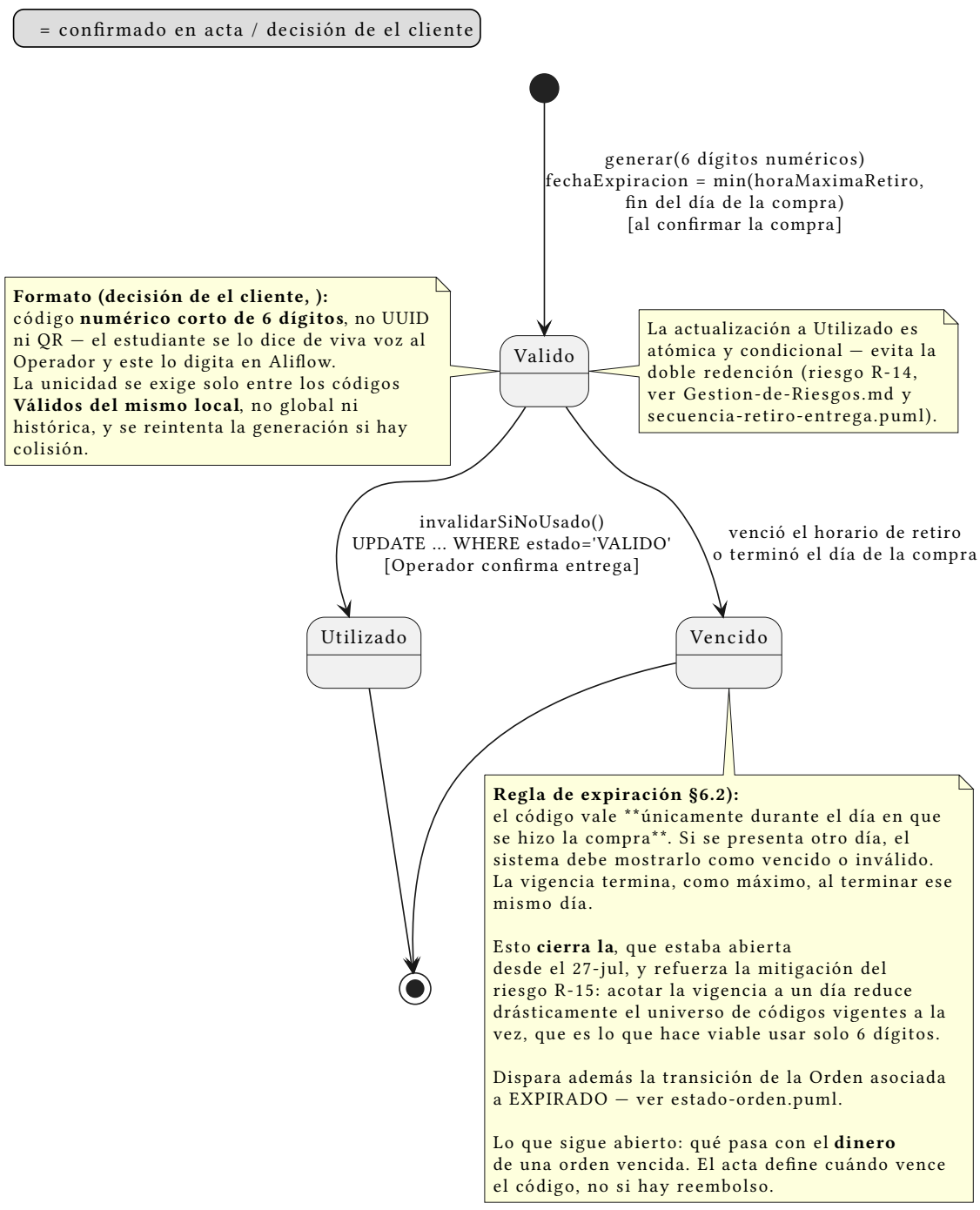


Figura 2: Estado: CodigoRetiro

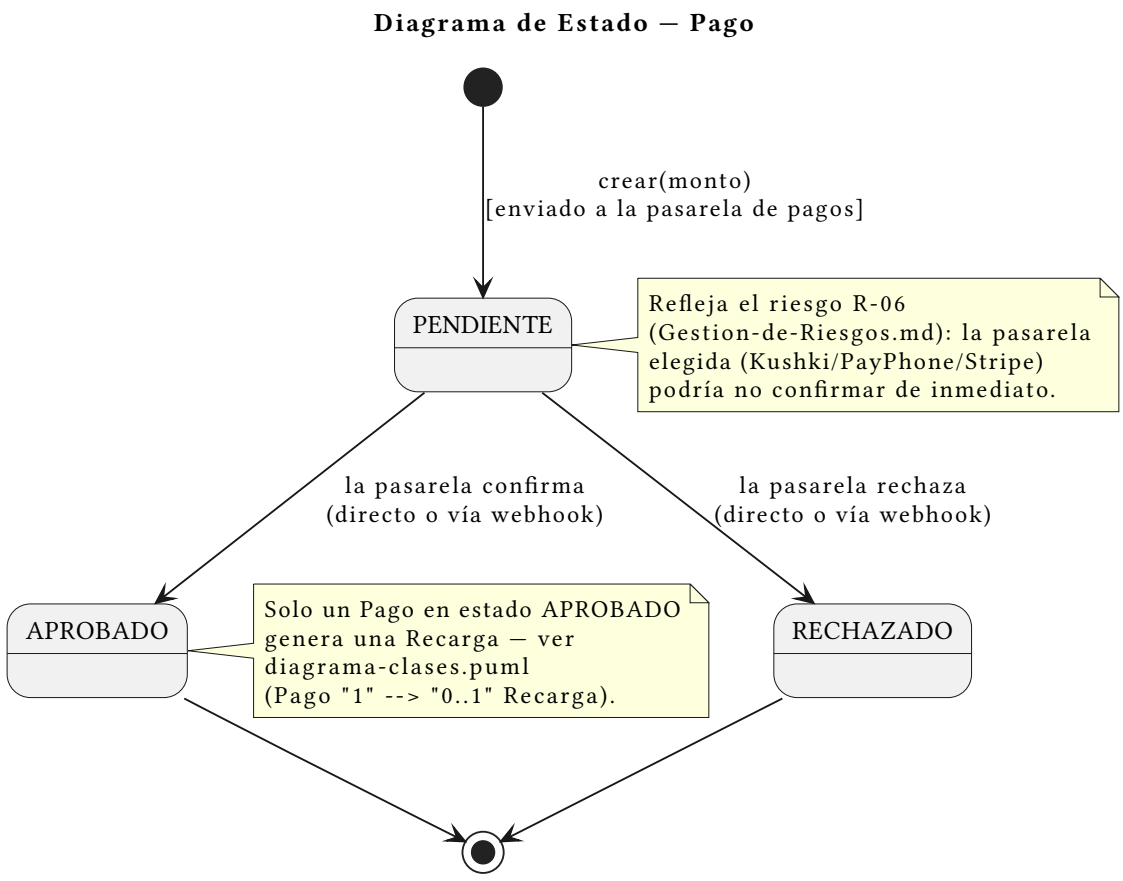


Figura 3: Estado: Pago

3. Pago

Fuente: `../../uml/estado-pago.puml` · **Referencia:** UC2, `../../uml/secuencia-recarga-saldo.puml`.

Simple pero real: `PENDIENTE` no es solo un estado transitorio instantáneo — puede persistir si la pasarela de pagos responde de forma asíncrona (riesgo R-06, `../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`). Solo `APROBADO` genera una `Recarga`.

4. EventoSincronizacion

Fuente: `../../uml/estado-evento-sincronizacion.puml` · **Referencia:** `../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4.

El auto-loop en `PENDIENTE` (reintento con backoff) y la transición a `FALLIDO` tras agotar intentos son exactamente el comportamiento ya mostrado en el diagrama de secuencia y en el diagrama de objetos con el caso real de Alpwin en Caramel Coffee — este diagrama es la vista de ciclo de vida del mismo mecanismo.

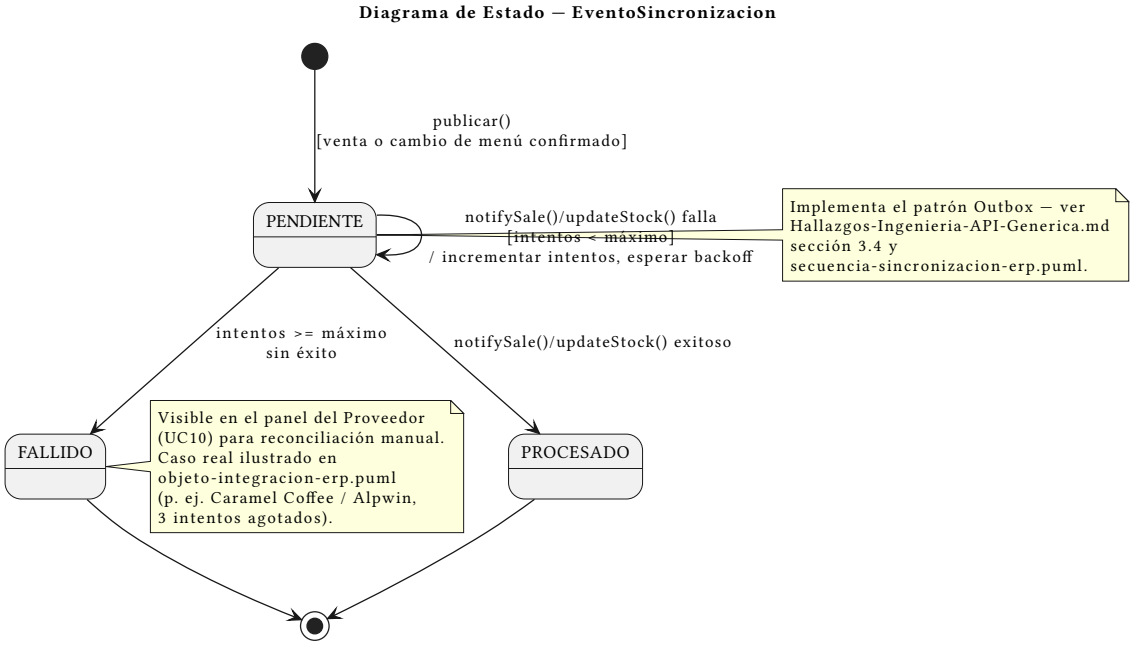


Figura 4: Estado: EventoSincronizacion

5. Cartilla de fidelidad

Fuente: `../../uml/estado-cartilla.puml` · **Referencia:** UC13/UC14/UC15, `../../uml/diagrama-clases.puml` paquete "Fidelidad".

Requisito nuevo, todo en amarillo: la cartilla de fidelidad. Es el diagrama de estado que más aporta de los cinco, porque el ciclo de vida **es** la regla de negocio — cuándo suma un sello, cuándo se puede canjear y cuándo se pierde.

Cuatro estados: `EN_CURSO` (acumulando), `COMPLETA` (premio disponible), `CANJEADA` y `EXPIRADA`. Dos detalles que no son obvios y por eso están anotados en el propio diagrama:

- **El auto-loop en `EN_CURSO` se dispara desde `marcarEntregado`, no desde `confirmarCompra`.** Una orden comprada y nunca retirada no suma sello. Sin esta regla, la cartilla se puede llenar sin ir nunca a buscar el almuerzo, y el local termina regalando un premio por ventas que no ocurrieron físicamente.
- **`COMPLETA → CANJEADA` es atómica y condicional** (`UPDATE... WHERE estado = 'COMPLETA'`), el mismo mecanismo que resuelve la doble redención del código de retiro (R-14). Sin esto, dos canjes simultáneos entregarían dos premios por una sola cartilla.

`EXPIRADA` es la parte más tentativa: el cliente no dijo si una cartilla a medio llenar caduca. Se modeló el estado con un campo `vigenciaCartillaDias` configurable; si la decisión es que no caduca, el campo queda nulo y la transición nunca se dispara — el modelo no cambia.

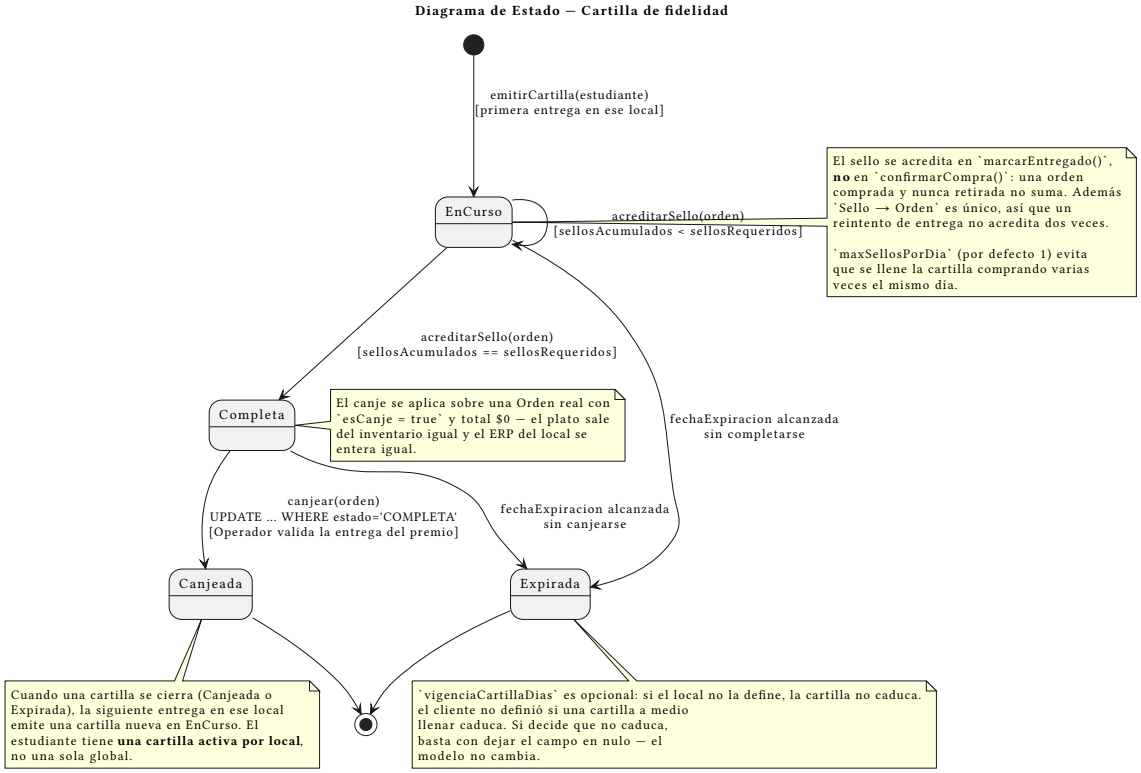


Figura 5: Estado: Cartilla

Por qué estos 5 y no otros

`Estudiante`, `Proveedor`, `Plato` y `ProgramaFidelidad` no tienen un ciclo de vida con estados discretos más allá de `activo`/`inactivo` (una sola transición booleana, no justifica un diagrama). `SaldoProveedor` y `TarjetaVirtual` cambian de valor (monto) pero no de "estado" en el sentido UML. Se priorizaron los objetos cuyo estado determina qué operaciones son válidas en cada momento.